

Guía de implementación de cámara, permisos y vibración

Evidencia de aprendizaje 3 · Aplicación móvil Android IU Digital Radio

Trabajo individual del rol simulado de Edwin Ruiz

Dato	Valor
Integrante	Edwin Ruiz
Rama asignada	feature/hardware
Responsabilidad	Cámara, permiso en tiempo de ejecución, vibración y pruebas de hardware
Base recibida	main con el estado y la interfaz principal ya integrados
Commit local verificado	37ede2d · Integra cámara permisos y vibración
Estado	Implementado, compilado y validado funcionalmente
Fecha de la guía	17 de septiembre de 2026

Propósito. Que Edwin pueda reproducir, comprender, probar y explicar su aporte sin depender de memorizar el código.

1. Resultado que debe entregar Edwin

Al finalizar, la aplicación debe solicitar el permiso de cámara cuando sea necesario, abrir la cámara del dispositivo, mostrar la fotografía capturada en la interfaz y emitir una vibración de confirmación después de una captura válida.

- La aplicación sigue funcionando si el dispositivo no tiene cámara; se muestra un mensaje explicativo.
- Si el usuario niega el permiso, la cámara no se abre y aparece un mensaje.
- Si el usuario cancela la captura, no se guarda una fotografía ni se produce la vibración.
- Si la captura es exitosa, el estado de Compose recibe el Bitmap, la fotografía se visualiza y luego se activa la confirmación háptica.
- El proyecto continúa compilando y las pruebas automatizadas verifican los permisos y la actualización del estado.

Importante. La vibración real debe comprobarse en un teléfono físico. Algunos emuladores no exponen un servicio de vibrador, aunque el código y las pruebas sean correctos.

2. Límite del trabajo de Edwin

Sí corresponde a Edwin	No corresponde a Edwin
Declarar CAMERA y VIBRATE en el manifiesto.	Rediseñar la pantalla principal de Juan Pablo.
Consultar y solicitar el permiso de cámara.	Cambiar las reglas del reducer creadas para el estado.
Abrir la cámara y recibir la miniatura capturada.	Implementar reproducción real de audio.
Activar la vibración tras una captura exitosa.	Hacer la integración final de todas las ramas.
Crear pruebas instrumentadas de configuración.	Subir directamente cambios a main sin revisión.

MainActivity usa temporalmente HardwareDemoRoute para que Edwin pueda ejecutar su módulo completo. En la integración final, Juan Ardila reemplazará ese coordinador provisional por RadioRoute y conservará la lógica probada.

3. Archivos involucrados

Archivo	Tipo	Finalidad
AndroidManifest.xml	Modificado	Permisos y declaración opcional de cámara.
CameraPermission.kt	Nuevo	Nombre y comprobación del permiso CAMERA.
DeviceVibrator.kt	Nuevo	Confirmación háptica de una captura exitosa.
HardwareDemoRoute.kt	Nuevo	Coordinación provisional entre UI, permiso, cámara y vibrador.
MainActivity.kt	Modificado	Entrada temporal al coordinador de hardware.
HardwareConfigurationTest.kt	Nuevo	Pruebas instrumentadas de permisos y fotografía en el estado.

4. Preparar la rama correctamente

Antes de crear archivos, Edwin debe recibir la última versión estable de main. Esto evita trabajar sobre una estructura antigua y reduce conflictos posteriores.

1. Abrir el proyecto iudigitalradioapp en Android Studio.
2. Abrir la terminal integrada en la carpeta raíz del proyecto.
3. Cambiar a main y traer la versión remota más reciente.
4. Cambiar a feature/hardware y combinar main dentro de esa rama.
5. Verificar que la terminal muestre feature/hardware y que el proyecto compile antes de comenzar.

```
git switch main
git pull origin main
git switch feature/hardware
git merge main
git status
```

Resultado esperado. Git debe indicar que Edwin está en feature/hardware. Si aparece un conflicto, no debe borrar archivos: debe avisar al coordinador y resolverlo comparando ambas versiones.

Estructura que debe crear

```
app/src/main/java/com/example/iudigitalradioapp/hardware/
├── CameraPermission.kt
├── DeviceVibrator.kt
└── HardwareDemoRoute.kt

app/src/androidTest/java/com/example/iudigitalradioapp/hardware/
└── HardwareConfigurationTest.kt
```

Cómo crear el package. En la vista Project de Android Studio, ubique `java/com.example.iudigitalradioapp`, haga clic derecho, seleccione `New > Package` y escriba `hardware`. Repita bajo `androidTest` para la prueba.

5. Declarar permisos y disponibilidad de cámara

Ruta: app/src/main/AndroidManifest.xml

El manifiesto informa a Android qué recursos de hardware utilizará la aplicación antes de que se ejecute.

Qué hace cada parte

- CAMERA permite solicitar acceso a la cámara. Es un permiso peligroso y, por eso, también debe pedirse durante la ejecución.
- VIBRATE autoriza la vibración. No necesita un cuadro de permiso en tiempo de ejecución.
- uses-feature con required=false evita excluir de la instalación a equipos sin cámara.
- El comentario de responsabilidad permite identificar el aporte de Edwin durante la revisión académica.

Código que debe quedar

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- Permisos del módulo de hardware asignado al rol simulado de Edwin Ruiz. -->
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.VIBRATE" />

    <!-- La aplicación puede instalarse aunque el dispositivo no tenga cámara. -->
    <uses-feature
        android:name="android.hardware.camera.any"
        android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Iudigitalradioapp">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.Iudigitalradioapp"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Evidencia recomendada. Captura del manifiesto mostrando las dos líneas uses-permission y el bloque uses-feature.

6. Centralizar el permiso de cámara

Ruta: app/src/main/java/com/example/iudigitalradioapp/hardware/CameraPermission.kt

Este objeto evita repetir el nombre del permiso y la consulta a ContextCompat en diferentes lugares.

Qué hace cada parte

- object crea una única instancia reutilizable; no es necesario construirla con new.
- permission conserva Manifest.permission.CAMERA como una constante legible.
- isGranted(context) compara el resultado de checkSelfPermission con PERMISSION_GRANTED.
- La clase no abre la cámara: solamente responde si el permiso ya fue concedido.

Código que debe quedar

```
package com.example.iudigitalradioapp.hardware

import android.Manifest
import android.content.Context
import android.content.pm.PackageManager
import androidx.core.content.ContextCompat

/**
 * Rol responsable en la simulación: Edwin Ruiz.
 *
 * Centraliza el nombre y la consulta del permiso de cámara. La solicitud se
 * realiza desde [HardwareDemoRoute] porque necesita un lanzador de actividad.
 */
object CameraPermission {

    const val permission: String = Manifest.permission.CAMERA

    fun isGranted(context: Context): Boolean {
        return ContextCompat.checkSelfPermission(
            context,
            permission
        ) == PackageManager.PERMISSION_GRANTED
    }
}
```

Cómo explicarlo. CameraPermission es una utilidad pequeña: concentra una responsabilidad y permite que HardwareDemoRoute tome la decisión correcta.

7. Implementar la confirmación por vibración

Ruta: app/src/main/java/com/example/iudigitalradioapp/hardware/DeviceVibrator.kt

Esta clase encapsula el acceso al servicio Vibrator y ofrece una función con un nombre relacionado con el caso de uso.

Qué hace cada parte

- applicationContext evita conservar accidentalmente una Activity más tiempo del necesario.
- getSystemService(Vibrator::class.java) obtiene el servicio del dispositivo; puede ser null si el equipo no lo ofrece.
- confirmPhotoCaptured usa EFFECT_DOUBLE_CLICK, una señal corta de confirmación.
- El operador ?. evita un error si no existe servicio de vibración.
- La función se invoca únicamente después de recibir una fotografía no nula.

Código que debe quedar

```
package com.example.iudigitalradioapp.hardware

import android.content.Context
import android.os.VibrationEffect
import android.os.Vibrator

/**
 * Rol responsable en la simulación: Edwin Ruiz.
 *
 * Envía una confirmación háptica breve después de capturar una fotografía. El
 * permiso VIBRATE es de instalación y se declara en AndroidManifest.xml.
 */
class DeviceVibrator(context: Context) {

    private val vibrator: Vibrator? = context.applicationContext
        .getSystemService(Vibrator::class.java)

    fun confirmPhotoCaptured() {
        vibrator?.vibrate(
            VibrationEffect.createPredefined(VibrationEffect.EFFECT_DOUBLE_CLICK)
        )
    }
}
```

Momento exacto de la vibración. Después de que la cámara devuelve un Bitmap válido y el reducer lo guarda en el estado. No vibra al abrir la cámara, pedir permiso, negar permiso ni cancelar la captura.

8. Coordinar permiso, cámara, estado y vibración

Ruta: app/src/main/java/com/example/iudigitalradioapp/hardware/HardwareDemoRoute.kt

HardwareDemoRoute es un composable provisional de integración. Conecta la pantalla existente con efectos del sistema Android.

Qué hace cada parte

- LocalContext obtiene el Context necesario para permisos, funciones del sistema y mensajes Toast.
- remember conserva DeviceVibrator y el estado mientras Compose recompone la pantalla.
- TakePicturePreview abre una aplicación de cámara compatible y devuelve un Bitmap pequeño o null si se cancela.
- RequestPermission muestra el diálogo del sistema y comunica si el permiso fue concedido.
- requestCamera comprueba primero que exista cámara, después revisa el permiso y finalmente abre o solicita lo necesario.
- RadioAction.OpenCamera se trata como un efecto externo; las demás acciones siguen pasando por RadioReducer.
- PhotoCaptured(bitmap) actualiza el estado antes de activar la vibración.

Código que debe quedar

```
package com.example.iudigitalradioapp.hardware

import android.content.pm.PackageManager
import android.widget.Toast
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.platform.LocalContext
import com.example.iudigitalradioapp.ui.RadioAction
import com.example.iudigitalradioapp.ui.RadioReducer
import com.example.iudigitalradioapp.ui.RadioScreen
import com.example.iudigitalradioapp.ui.createInitialRadioUiState

/**
 * Rol responsable en la simulación: Edwin Ruiz.
 *
 * Coordinador provisional usado para comprobar cámara, permiso y vibración.
 * Juan Ardila trasladará estos efectos a RadioRoute durante la integración.
 */
@Composable
fun HardwareDemoRoute() {
    val context = LocalContext.current
    val deviceVibrator = remember(context) {
        DeviceVibrator(context)
    }
    var state by remember {
        mutableStateOf(createInitialRadioUiState())
    }

    val cameraLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.TakePicturePreview()
    ) { bitmap ->
        if (bitmap == null) {
            Toast.makeText(
                context,
                "La captura fue cancelada.",
                Toast.LENGTH_SHORT
            ).show()
        } else {
            state = RadioReducer.reduce(
                state = state,
                action = RadioAction.PhotoCaptured(bitmap)
            )
            deviceVibrator.confirmPhotoCaptured()
        }
    }
}
```

```

val permissionLauncher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.RequestPermission()
) { granted ->
    if (granted) {
        cameraLauncher.launch(null)
    } else {
        Toast.makeText(
            context,
            "El permiso de cámara es necesario para tomar la fotografía.",
            Toast.LENGTH_LONG
        ).show()
    }
}

fun requestCamera() {
    val hasCamera = context.packageManager.hasSystemFeature(
        PackageManager.FEATURE_CAMERA_ANY
    )

    when {
        !hasCamera -> {
            Toast.makeText(
                context,
                "Este dispositivo no tiene una cámara disponible.",
                Toast.LENGTH_LONG
            ).show()
        }

        CameraPermission.isGranted(context) -> {
            cameraLauncher.launch(null)
        }

        else -> {
            permissionLauncher.launch(CameraPermission.permission)
        }
    }
}

RadioScreen(
    state = state,
    onAction = { action ->
        if (action == RadioAction.OpenCamera) {
            requestCamera()
        } else {
            state = RadioReducer.reduce(state, action)
        }
    }
)
}

```

Flujo funcional

```

Usuario pulsa Abrir cámara
↓
¿El dispositivo tiene cámara?
├─ No → mostrar mensaje y terminar
└─ Sí
    ↓
    ¿CAMERA ya está concedido?
    ├─ No → solicitar permiso
    │   └─ Negado → mostrar mensaje
    │       └─ Concedido → abrir cámara
    └─ Sí → abrir cámara
        ↓
        ¿La cámara devolvió una fotografía?
        ├─ No → informar cancelación
        └─ Sí → guardar Bitmap → mostrarlo → vibrar
    
```

9. Conectar temporalmente el módulo

Ruta: app/src/main/java/com/example/iudigitalradioapp/MainActivity.kt

La Activity instala el tema de Compose y presenta HardwareDemoRoute para probar el aporte de Edwin de principio a fin.

Qué hace cada parte

- setContent inicia la interfaz declarativa.
- IudigitalradioappTheme conserva los estilos del proyecto.
- HardwareDemoRoute es temporal; permite validar la rama sin esperar la integración final.
- El comentario deja explícito que Juan Ardila hará el reemplazo por RadioRoute.

Código que debe quedar

```
package com.example.iudigitalradioapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.iudigitalradioapp.hardware.HardwareDemoRoute
import com.example.iudigitalradioapp.ui.theme.IudigitalradioappTheme

/**
 * Punto de entrada de la aplicación.
 *
 * La conexión temporal con [HardwareDemoRoute] permite que Edwin Ruiz pruebe
 * la cámara, el permiso y la vibración sobre la interfaz existente. Juan
 * Ardila reemplazará esta conexión por RadioRoute en la integración final.
 */
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            IudigitalradioappTheme {
                HardwareDemoRoute()
            }
        }
    }
}
```

No confundir. Edwin no está apropiándose de la integración final. Esta conexión es una plataforma de prueba temporal y está documentada como tal.

10. Crear las pruebas instrumentadas

Ruta:

app/src/androidTest/java/com/example/iudigitalradioapp/hardware/HardwareConfigurationTest.kt

Las pruebas instrumentadas se ejecutan en Android porque necesitan PackageManager y Bitmap reales del framework.

Qué hace cada parte

- manifestDeclaresCameraAndVibrationPermissions inspecciona el paquete instalado y confirma que ambos permisos fueron solicitados.
- La condición por versión usa PackageInfoFlags en Android 13 o superior y conserva compatibilidad con versiones anteriores.
- capturedPhotoIsStoredInScreenState crea un Bitmap pequeño, envía PhotoCaptured al reducir y comprueba que se conserva la misma instancia.
- Estas pruebas no sustituyen la comprobación manual del diálogo, la cámara ni la vibración física.

Código que debe quedar

```
package com.example.iudigitalradioapp.hardware

import android.Manifest
import android.content.pm.PackageManager
import android.graphics.Bitmap
import android.os.Build
import androidx.test.ext.junit.runners.AndroidJUnit4
import androidx.test.platform.app.InstrumentationRegistry
import com.example.iudigitalradioapp.ui.RadioAction
import com.example.iudigitalradioapp.ui.RadioReducer
import com.example.iudigitalradioapp.ui.createInitialRadioUiState
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test
import org.junit.runner.RunWith

/** Pruebas instrumentadas del rol simulado de Edwin Ruiz. */
@RunWith(AndroidJUnit4::class)
class HardwareConfigurationTest {

    @Test
    fun manifestDeclaresCameraAndVibrationPermissions() {
        val context = InstrumentationRegistry.getInstrumentation().targetContext
        val packageInfo = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            context.packageManager.getPackageInfo(
                context.packageName,
                PackageManager.PackageInfoFlags.of(PackageManager.GET_PERMISSIONS.toLong())
            )
        } else {
            @Suppress("DEPRECATION")
            context.packageManager.getPackageInfo(
                context.packageName,
                PackageManager.GET_PERMISSIONS
            )
        }
        val permissions = packageInfo.requestedPermissions.orEmpty().toSet()

        assertTrue(Manifest.permission.CAMERA in permissions)
        assertTrue(Manifest.permission.VIBRATE in permissions)
    }

    @Test
    fun capturedPhotoIsStoredInScreenState() {
        val bitmap = Bitmap.createBitmap(2, 2, Bitmap.Config.ARGB_8888)

        val result = RadioReducer.reduce(
            state = createInitialRadioUiState(),
            action = RadioAction.PhotoCaptured(bitmap)
        )

        assertEquals(bitmap, result.capturedPhoto)
    }
}
```


11. Ejecutar y probar manualmente

11.1 Preparar la primera solicitud

Si el permiso ya se había concedido, el cuadro de Android no volverá a aparecer. Para documentar el primer uso, se puede revocar desde Ajustes > Aplicaciones > IU Digital Radio > Permisos > Cámara > No permitir, y luego abrir otra vez la aplicación.

Alternativa con ADB. Este comando es opcional y se ejecuta con un dispositivo o emulador conectado:

```
adb shell pm revoke com.example.iudigitalradioapp android.permission.CAMERA
```

11.2 Casos manuales

Caso	Acción	Resultado esperado
Permiso pendiente	Pulsar Abrir cámara.	Android solicita autorización para usar la cámara.
Permiso negado	Seleccionar No permitir.	La cámara no abre y aparece el mensaje explicativo.
Permiso concedido	Autorizar y continuar.	Se abre la aplicación de cámara disponible.
Captura cancelada	Salir de la cámara sin aceptar foto.	Se informa la cancelación; no hay foto nueva ni vibración.
Captura válida	Tomar y aceptar una foto.	La imagen aparece en la tarjeta y el teléfono vibra dos pulsos breves.
Sin cámara	Ejecutar en un equipo que no la exponga.	Se muestra que no hay cámara disponible sin cerrar la aplicación.

Prueba ya realizada. El flujo de cámara, permiso y actualización de la fotografía fue probado correctamente. La vibración debe registrarse desde un teléfono físico porque el emulador empleado no ofrece servicio de vibrador.

12. Verificación técnica realizada

Desde la raíz del proyecto se ejecutaron las siguientes tareas de Gradle:

```
.\gradlew.bat testDebugUnitTest assembleDebug compileDebugAndroidTestKotlin
.\gradlew.bat connectedDebugAndroidTest
.\gradlew.bat lintDebug
```

Verificación	Resultado
Pruebas unitarias	12 aprobadas.
Pruebas instrumentadas	6 aprobadas: 1 base, 3 de interfaz y 2 de hardware.
APK de depuración	assembleDebug finalizó correctamente.
Compilación de androidTest	compileDebugAndroidTestKotlin finalizó correctamente.
Análisis estático	lintDebug finalizó con BUILD SUCCESSFUL.
Advertencias de lint	15 no bloqueantes: dependencias disponibles, colores de plantilla sin usar y label redundante preexistente.

13. Revisar y crear el commit

Edwin debe confirmar que su commit incluya solamente los archivos funcionales del módulo. Los directorios .idea, output y tmp no forman parte del aporte.

```
git status
git diff -- app/src/main/AndroidManifest.xml
git diff -- app/src/main/java/com/example/iudigitalradioapp/MainActivity.kt
git diff -- app/src/main/java/com/example/iudigitalradioapp/hardware
git diff -- app/src/androidTest/java/com/example/iudigitalradioapp/hardware
```

Si el contenido es correcto, se agregan rutas explícitas para evitar subir archivos locales por accidente:

```
git add app/src/main/AndroidManifest.xml
git add app/src/main/java/com/example/iudigitalradioapp/MainActivity.kt
git add app/src/main/java/com/example/iudigitalradioapp/hardware
git add app/src/androidTest/java/com/example/iudigitalradioapp/hardware
git commit -m "Integra cámara permisos y vibración"
git status
```

Commit local actual. La implementación validada quedó registrada como 37ede2d con el mensaje “Integra cámara permisos y vibración”.

14. Publicar la rama y solicitar Pull Request

1. Publicar feature/hardware en el repositorio remoto.
2. Abrir GitHub y seleccionar Compare & pull request.
3. Confirmar base: main y compare: feature/hardware.
4. Escribir un resumen corto y las pruebas realizadas.
5. Asignar la revisión a Juan Ardila, responsable de integración.
6. Esperar aprobación antes de combinar; Edwin no debe hacer push directo a main.

```
git push -u origin feature/hardware
```

Texto sugerido para el Pull Request

Título: Integra cámara, permisos y vibración

Resumen:

- declara CAMERA y VIBRATE en el manifiesto;
- solicita CAMERA durante la ejecución;
- abre la cámara con TakePicturePreview;
- muestra la captura en el estado de la pantalla;
- vibra después de una captura válida;
- agrega pruebas instrumentadas de configuración.

Validación:

- 12 pruebas unitarias aprobadas;
- 6 pruebas instrumentadas aprobadas;
- assembleDebug y lintDebug correctos;
- flujo manual de cámara probado.

Después del merge. Juan Ardila debe actualizar su main, ejecutar nuevamente las pruebas y sustituir HardwareDemoRoute por RadioRoute durante la integración general.

15. Empaquetar una copia ZIP opcional

GitHub y el Pull Request son la evidencia principal del trabajo colaborativo. Si el docente también solicita un ZIP, se debe generar sin carpetas temporales ni configuración local.

1. Cerrar la aplicación si está ejecutándose, pero no es obligatorio cerrar Android Studio.
2. En GitHub, abrir la rama feature/hardware y usar Code > Download ZIP; así solo se incluyen archivos rastreados.
3. Renombrar el archivo como iudigitalradioapp-feature-hardware-Edwin-Ruiz.zip.
4. Abrir el ZIP y verificar que contenga app, gradle, build.gradle.kts, settings.gradle.kts y gradlew.bat.

No incluir. .idea, .gradle, build, output, tmp, archivos de emulador ni credenciales locales.

16. Capturas numeradas para la evidencia

Las capturas son evidencia de lo realizado y de que funciona. No necesitan registrar cada clic de creación. Deben verse claras, estar numeradas y llevar una explicación debajo.

N.º	Captura	Texto sugerido debajo
1	Terminal en feature/hardware después de integrar main.	Se preparó la rama individual a partir de la versión estable del equipo.
2	Árbol de paquetes con la carpeta hardware y sus tres archivos.	Se organizó el acceso a hardware en un package con responsabilidad definida.
3	AndroidManifest.xml con CAMERA, VIBRATE y uses-feature.	Se declararon permisos y se mantuvo opcional la disponibilidad de cámara.
4	Código de CameraPermission y DeviceVibrator.	Se centralizó la consulta del permiso y la confirmación háptica.
5	Diálogo real de Android solicitando cámara.	La aplicación solicita el permiso peligroso durante la ejecución.
6	Aplicación de cámara abierta desde IU Digital Radio.	Tras autorizar, el contrato de actividad inicia la captura.
7	Pantalla mostrando la fotografía capturada.	El Bitmap devuelto se almacenó en el estado y Compose actualizó la interfaz.
8	Teléfono físico o nota de prueba de vibración.	Después de aceptar la foto se emitió la confirmación de doble pulsación.
9	Salida BUILD SUCCESSFUL de las pruebas.	Las pruebas unitarias e instrumentadas finalizaron correctamente.
10	Historial de Git con el commit de Edwin.	El aporte quedó aislado en un commit identificable de feature/hardware.
11	Pull Request feature/hardware → main.	El trabajo se envió a revisión antes de integrarse a la rama principal.
12	Pull Request aprobado o main actualizado.	El coordinador revisó e incorporó el módulo al producto común.

Consejo de presentación. Recorte únicamente información ajena, conserve visible la rama o el nombre del archivo y no edite mensajes para aparentar resultados distintos de los obtenidos.

17. Lista de comprobación de Edwin

- ☐ Estoy ubicado en feature/hardware.
- ☐ AndroidManifest.xml declara CAMERA, VIBRATE y cámara opcional.
- ☐ CameraPermission.kt comprueba el permiso sin abrir la cámara.
- ☐ DeviceVibrator.kt vibra solamente tras una captura válida.
- ☐ HardwareDemoRoute.kt cubre cámara ausente, permiso, cancelación y éxito.
- ☐ MainActivity.kt se reconoce como conexión temporal.
- ☐ Las pruebas unitarias e instrumentadas terminan correctamente.
- ☐ Probé el flujo manual y registré capturas numeradas.
- ☐ Mi commit no incluye .idea, output, tmp ni carpetas build.
- ☐ Publiqué feature/hardware y abrí un Pull Request hacia main.

18. Preguntas para sustentar el trabajo

Pregunta	Respuesta breve esperada
¿Por qué CAMERA se pide durante la ejecución?	Porque Android lo clasifica como permiso peligroso y el usuario debe decidir al momento de usar la función.
¿Por qué VIBRATE no muestra diálogo?	Se concede al instalar; basta declararlo en el manifiesto.
¿Qué devuelve TakePicturePreview?	Un Bitmap pequeño si la captura se acepta o null si se cancela.
¿Cuándo vibra la aplicación?	Después de guardar en el estado una fotografía válida.
¿Qué ocurre si no hay cámara?	Se muestra un Toast y la aplicación continúa abierta.
¿Por qué DeviceVibrator acepta Context?	Para obtener el servicio Android sin acoplar la lógica directamente al composable.
¿Para qué sirve remember?	Mantiene objetos y estado entre recomposiciones de Compose.
¿Por qué HardwareDemoRoute es provisional?	Permite probar la rama; RadioRoute será el coordinador definitivo de todos los módulos.
¿Qué prueban las pruebas instrumentadas?	Que el APK declara ambos permisos y que PhotoCaptured conserva el Bitmap en el estado.
¿Por qué usar Pull Request?	Para revisar, documentar y combinar el aporte sin modificar main directamente.

19. Referencias

Android Developers. (s. f.). Request runtime permissions.

<https://developer.android.com/training/permissions/requesting>

Android Developers. (s. f.). ActivityResultContracts.TakePicturePreview.

<https://developer.android.com/reference/androidx/activity/result/contract/ActivityResultContracts.TakePicturePreview>

Android Developers. (s. f.). Add haptic feedback to events.

<https://developer.android.com/develop/ui/views/haptics/haptic-feedback>

GitHub Docs. (s. f.). Creating a pull request. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request>

20. Cierre

El aporte de Edwin queda completo cuando el código está en feature/hardware, las pruebas pasan, el flujo manual está documentado y existe un Pull Request revisable hacia main. El objetivo no es solamente que la cámara se abra, sino poder explicar la relación entre permiso, contrato de actividad, estado de Compose y respuesta háptica.